Perencanaan Pembuatan – KelolaSupport E-Ticket (Web, Next.js)

1. Ringkasan Proyek

KelolaSupport adalah aplikasi web helpdesk/e-ticket untuk client mengajukan keluhan atau pertanyaan, dijawab pertama kali oleh **chatbot**, dan bila client belum puas bisa **eskalasi ke live chat dengan petugas yang sedang online**.

2. Tujuan

1. Client bisa membuat tiket dan langsung dapat respons awal dari bot (tanpa menunggu agen).
2. Kalau bot tidak memuaskan, client bisa klik "Bicara dengan Petugas" → tiket dialihkan ke agen yang online.
3. Petugas (Customer Support) mengelola antrean tiket, chat real-time dengan client, dan mengubah status tiket.

4. Admin mengelola user, role, shift, kategori, SLA, dan melihat dashboard performa.
5. Sistem tercatat rapi: SLA, audit log, laporan — walau skala PKL, tetap dibuat "production-like".

3. Ruang Lingkup

In Scope (dikerjakan)

- Web app (client-facing + petugas + admin) dalam 1 aplikasi Next.js (multi-role, route-protected).
- Autentikasi & role-based access (Admin, Petugas, Client).
- Tiket: create, kategori, priority, status, timeline.
- Chatbot rule-based/FAQ-matching dulu (bisa upgrade ke LLM API belakangan — arsitektur disiapkan).
- Eskalasi chatbot → live chat petugas (real-time, pakai WebSocket/polling).
- Shift & status online/offline petugas → dasar auto-assign.
- SLA dasar (target waktu respon/selesai per priority) + notifikasi in-app.

- Dashboard & laporan sederhana (jumlah tiket, rata-rata respon, performa petugas).
- Audit log aksi penting.

4. Role & Aktor

Role	Deskripsi Singkat
Admin	Kelola user/role, kategori, SLA, shift, lihat dashboard & laporan seluruh sistem.
Petugas L1 (Customer Support)	Terima tiket hasil eskalasi dari bot, **satu-satunya pihak petugas yang chat langsung dengan client** (Public Chat). Selesaikan masalah dasar/umum sendiri. Kalau masalah bersifat teknis, buka **Internal Chat** dengan Petugas L2, lalu setelah dapat solusi, L1 yang menyampaikan ke client.
Petugas L2 (Technical Support)	**Tidak chat dengan client sama sekali.** Hanya berkomunikasi dengan Petugas L1 lewat Internal Chat untuk membantu analisis & memperbaiki masalah teknis (bug, error aplikasi, kerusakan

Role	Deskripsi Singkat
	sistem). Solusi disampaikan ke L1, bukan langsung ke client.
Client	Buat tiket, chat dengan bot dulu, lalu bisa minta lanjut ke petugas manusia (selalu lewat L1).

- **L1 – Customer Support**: baris depan setelah bot, dan **satu-satunya jembatan komunikasi ke client**. Punya 2 kanal chat di halaman tiket: **Public Chat** (dengan client) dan **Internal Chat** (dengan L2).
- **L2 – Technical Support**: backend murni. Hanya punya **Internal Chat** (dengan L1), tidak ada akses Public Chat ke client sama sekali.
- Jadi total ada **2 jenis eskalasi**: (1) Bot → Petugas L1 (client tidak puas jawaban bot), dan (2) Petugas L1 → Petugas L2 (masalah teknis) – tapi eskalasi ke L2 ini **tidak memindahkan kepemilikan komunikasi ke client**; L1 tetap pegang Public Chat, L2 hanya membantu di baliknya lewat Internal Chat.

5. Alur Utama (High-Level)

1. Client login/register → buat tiket baru (judul, kategori, deskripsi, priority otomatis/manual).

2. Sistem buka sesi chat, bot langsung merespons berdasar kata kunci/kategori tiket.
3. Client puas → tiket bisa langsung ditutup / auto-close setelah beberapa waktu tanpa balasan.
4. Client tidak puas → klik "Chat dengan Petugas" → sistem cari **Petugas L1** yang online sesuai shift → auto-assign (atau masuk antrean kalau semua offline).
5. Petugas L1 menangani via **Public Chat** dengan client, bisa update status, lampirkan file.
6. Kalau masalah ternyata teknis (bug/error aplikasi) → Petugas L1 membuka **Internal Chat** dengan Petugas L2 yang online. L2 membantu analisis & investigasi lewat Internal Chat ini saja (tidak menyentuh Public Chat).
7. Setelah dapat solusi dari L2, **Petugas L1** yang menyampaikan ke client via Public Chat, lalu menandai tiket selesai.
8. Tiket selesai → ditutup, SLA dicatat, masuk laporan.

6. Tech Stack (usulan)

Layer	Teknologi	Alasan
Frontend + Backend	**Next.js (App Router)**	Full-stack 1 framework, cocok untuk role-based page & API routes.
Bahasa	TypeScript	Type-safety untuk struktur role/ticket yang kompleks.
Styling	Tailwind CSS	Cepat, konsisten dengan skill desain yang ada.
Database	PostgreSQL	Relasional, cocok untuk relasi ticket-user-role-sla.

Layer	Teknologi	Alasan
ORM	Prisma	Migrasi & query type-safe, cocok dengan Next.js.
Auth	NextAuth.js / Auth.js (credentials + JWT)	Role-based session management.
Realtime chat	WebSocket (Socket.IO) atau Pusher/Ably	Live chat client ↔ bot ↔ petugas.
Bot logic	Rule-based intent matching (V1), API terpisah agar mudah upgrade ke LLM	Sesuai arahan "arsitektur disiapkan untuk AI".
Deployment	Vercel (app) + managed Postgres (Neon/Supabase)	Gratis/murah untuk skala PKL.

7. Modul/Fitur Tingkat Tinggi (detail lengkap di Tahap 2)

1. Auth & Role Management
2. Manajemen User & Shift (Admin)
3. Tiket (create, kategori, priority, status, timeline)
4. Chat: Bot Chat → Eskalasi ke Petugas L1 → Eskalasi ke Petugas L2 (teknis)
5. Assignment & Antrean Petugas Online (per level, sesuai shift)
6. SLA & Notifikasi
7. Dashboard & Laporan
8. Audit Log

8. Tahapan Pengerjaan (Roadmap Proses, Bukan Waktu Coding)

Tahap	Isi	Status
1	Perencanaan Pembuatan (dokumen ini)	✅ Selesai
2	Analisis Fitur (detail per modul, requirement, business rules)	⏭ Berikutnya
3	Desain Diagram (Flowchart, ERD, Use Case Diagram, Activity	Menunggu

Tahap	Isi	Status
	Diagram, Sequence Diagram chat)	
4	Setup Project Next.js + implementasi bertahap per modul	Menunggu

9. Deliverable Tiap Tahap

- Tahap 1: Dokumen perencanaan ini.
- Tahap 2: Dokumen analisis fitur + daftar requirement (functional & non-functional).
- Tahap 3: Kumpulan diagram (bisa dalam bentuk file Mermaid/gambar) + penjelasan.
- Tahap 4: Source code Next.js siap jalan (dengan struktur folder jelas per role).
